ASSIGNMENT 18.4

Shaik Sabina

2303A51303

BATCH – 05

TASK – 01

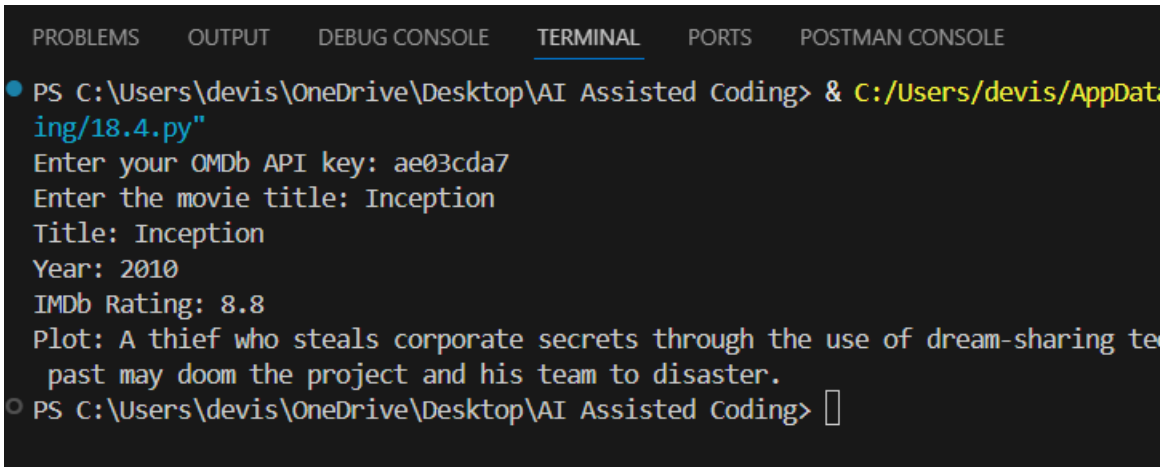
Prompt : Create a Python script using the requests library to fetch movie details from OMDb API.Requirements: Take movie title as user input, Fetch data from API using API key,Display: Title, Year, IMDb Rating, Plot,Handle errors: invalid movie, API key issues, network errors, empty response,Use clean formatting and functions

Code :

```
#Create a Python script using the requests library to fetch movie details from OMDb API.
#Requirements: Take movie title as user input, Fetch data from API using API key,Display: Title, Year, IMDb Rating, Plot,Handle errors: invalid movie, API key issues, network errors, empty response,Use clean formatting and functions
from gettext import install
import requests
def fetch_movie_details(movie_title, api_key):
    url = f"http://www.omdbapi.com/?t={movie_title}&apikey={api_key}"
    try:
        response = requests.get(url)
        response.raise_for_status()  # Check for HTTP errors
        data = response.json()

        if data['Response'] == 'False':
            print(f"Error: {data['Error']}")
            return

        print(f"Title: {data.get('Title', 'N/A')}")
        print(f"Year: {data.get('Year', 'N/A')}")
        print(f"IMDb Rating: {data.get('imdbRating', 'N/A')}")
        print(f"Plot: {data.get('Plot', 'N/A')}")

    except requests.exceptions.RequestException as e:
        print(f"Network error: {e}")
    except ValueError:
        print("Error parsing response.")
def main():
    api_key = input("Enter your OMDb API key: ")
    movie_title = input("Enter the movie title: ")
```

```
    fetch_movie_details(movie_title, api_key)
if __name__== "__main__":
    main()
```

Output :



Explanation : This script allows users to fetch movie details from the OMDb API by entering a movie title and an API key. It uses the requests library to make an HTTP GET request to the API, retrieves the movie details in JSON format, and displays the title, year, IMDb rating, and plot. The script also includes error handling for invalid movie titles, API key issues, network errors, and empty responses.

TASK – 02

Prompt : Create a Python script using the requests library to fetch public transport arrival times from an API. Take stop ID as user input, validate it, and display the next 3 arrivals. Handle errors like invalid stop ID, server issues, timeout, and empty response using try-except. Include at least one retry attempt and keep the output clean and readable.

Code :

```
# Create a Python script using the requests library to fetch public transport
arrival times from an API. Take stop ID as user input, validate it, and
display the next 3 arrivals. Handle errors like invalid stop ID, server
issues, timeout, and empty response using try-except. Include at least one
retry attempt and keep the output clean and readable.
import requests
import time
```

```python
def fetch_arrival_times(stop_id, retries=1):
    url = f"https://api.tfl.gov.uk/StopPoint/{stop_id}/Arrivals"

    attempt = 0

    while attempt <= retries:
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()

            data = response.json()

            # Check empty response
            if not data:
                print("❌ No arrival data found for this stop.")
                return

            # Sort by soonest arrival
            data.sort(key=lambda x: x.get("timeToStation", 0))

            print(f"\n🚉 Next 3 arrivals for stop ID {stop_id}:")
            print("-" * 40)

            for arrival in data[:3]:
                line = arrival.get("lineName", "N/A")
                destination = arrival.get("destinationName", "N/A")
                time_sec = arrival.get("timeToStation", 0)

                minutes = time_sec // 60

                print(f"Line {line} → {destination} in {minutes} min")

            print("-" * 40)
            return

        except requests.exceptions.Timeout: print("⏱
            Timeout error. Retrying...")
        except requests.exceptions.ConnectionError:
            print("🌐 Connection error. Retrying...")
        except requests.exceptions.HTTPError as http_err:
            print(f"❌ HTTP error: {http_err}")
            return
        except requests.exceptions.RequestException as e:
            print(f"⚠️ Error: {e}")
            return

        attempt += 1
        time.sleep(2)  # small delay before retry
```

```
    print("❌ Failed after retrying.")


def main():
    stop_id = input("Enter the stop ID: ").strip()

    if not stop_id:
        print("❌ Invalid stop ID.")
        return

    fetch_arrival_times(stop_id)


if __name__ == "__main__":
    main()
```

Output :

```
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding> & C:/U
ing/18.4.py"
Enter the stop ID: 490008660N

🚏 Next 3 arrivals for stop ID 490008660N:
---------------------------------------
Line 88 → Parliament Hill Fields in 4 min
Line 214 → Highgate Village in 19 min
---------------------------------------
PS C:\Users\devis\OneDrive\Desktop\AI Assisted Coding>
```

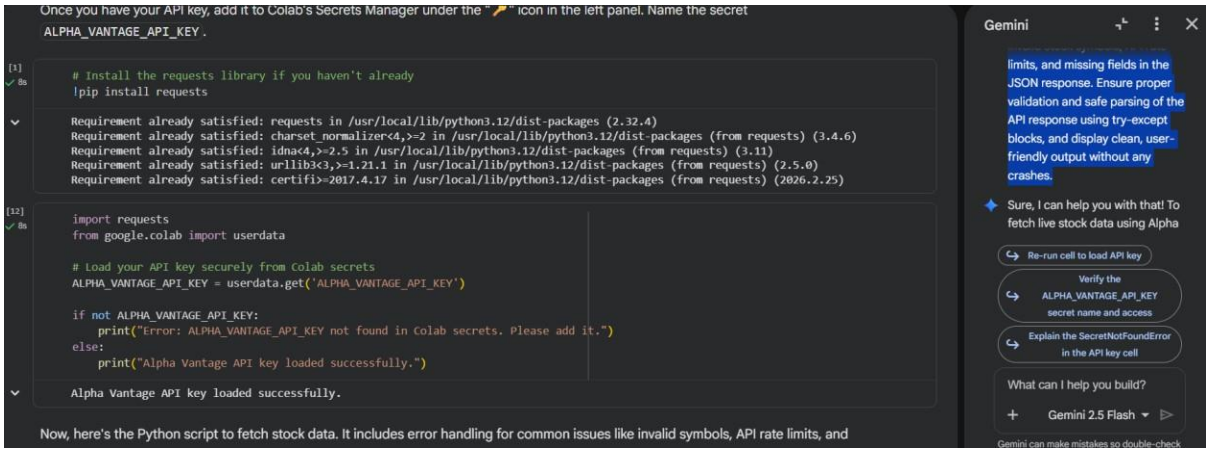
Explanation : The program takes a stop ID as input and fetches arrival data from the TfL API using the requests library.
It processes the response, sorts arrivals by nearest time, and displays the next 3 arrivals with details.
It also handles errors (timeout, connection, invalid data) and includes a retry mechanism for reliability**.**

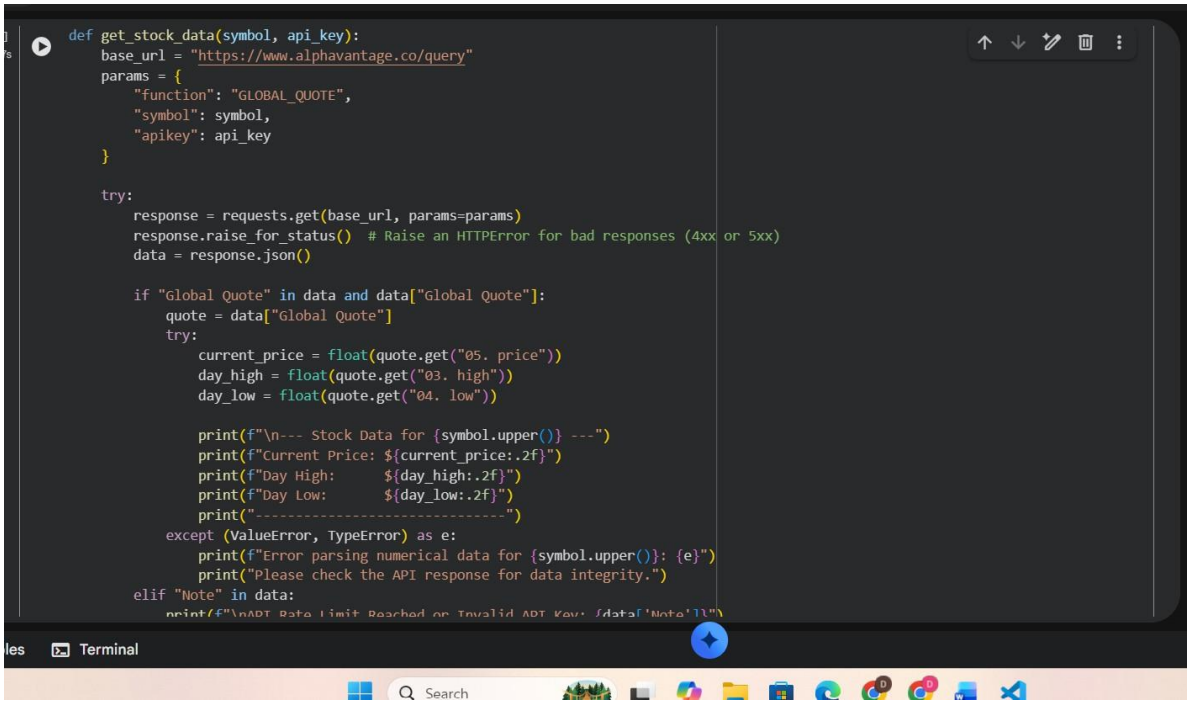
TASK – 03

Prompt : Create a Python script using the requests library to fetch live stock data from a financial API (such as Alpha Vantage). The program should accept a stock symbol as user input and display the current price, day high, and day low. It must handle errors like invalid stock symbols, API rate limits, and missing fields in the JSON response. Ensure proper validation and safe parsing of the

API response using try-except blocks, and display clean, user-friendly output without any crashes.

Code :





```
            print(f"\nAPI Rate Limit Reached or Invalid API Key: {data['Note']}")
            print("Please wait a minute and try again, or check your API key.")
        elif "Error Message" in data:
            print(f"\nError for symbol {symbol.upper()}: {data['Error Message']}")
            print("This might be an invalid stock symbol or a problem with the request.")
        else:
            print(f"\nCould not retrieve data for {symbol.upper()}. Unknown API response format.")
            print("Full API response:", data)

    except requests.exceptions.HTTPError as errh:
        print(f"HTTP Error: {errh}")
    except requests.exceptions.ConnectionError as errc:
        print(f"Error Connecting: {errc}")
    except requests.exceptions.Timeout as errt:
        print(f"Timeout Error: {errt}")
    except requests.exceptions.RequestException as err:
        print(f"An unexpected error occurred: {err}")
    except ValueError as e:
        print(f"Error decoding JSON response: {e}")

# Get stock symbol from user input
if ALPHA_VANTAGE_API_KEY:
    stock_symbol = input("Enter a stock symbol (e.g., AAPL, GOOGL): ").strip().upper()
    if stock_symbol:
        get_stock_data(stock_symbol, ALPHA_VANTAGE_API_KEY)
    else:
        print("No stock symbol entered.")
else:
    print("API key is not configured. Please ensure ALPHA_VANTAGE_API_KEY is set in Colab secrets.")
```

Output :

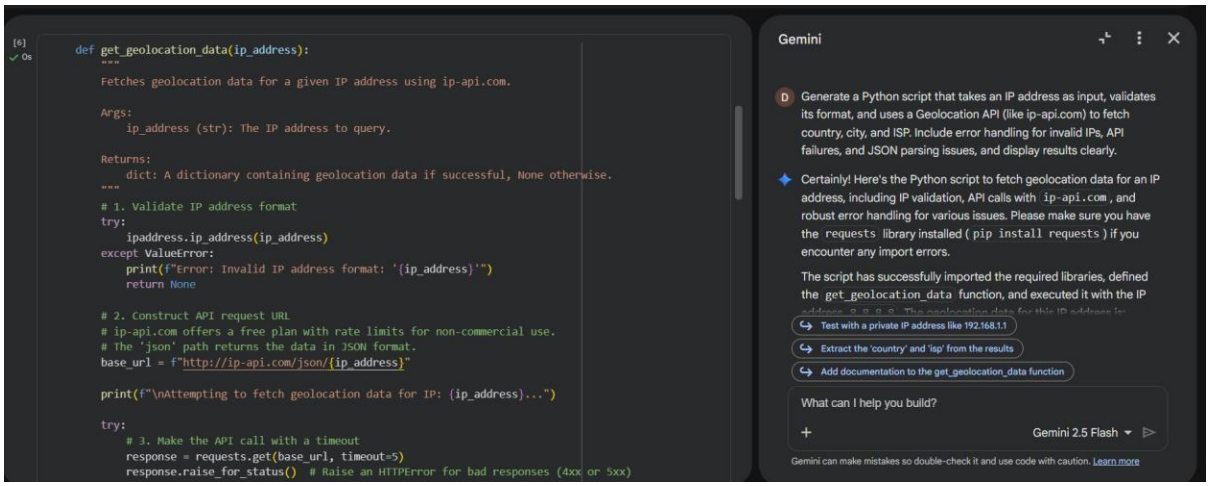
```
...  Enter a stock symbol (e.g., AAPL, GOOGL): AAPL

    --- Stock Data for AAPL ---
    Current Price: $252.62
    Day High:      $255.00
    Day Low:       $251.60
    ------------------------------
```

Explanation: The script takes a stock symbol as input and fetches its current price, day high, and day low from a financial API.
It safely handles errors like invalid symbols, rate limits, and missing data, ensuring clean output without crashes.

TASK – 04:

Prompt : Generate a Python script that takes an IP address as input, validates its format, and uses a Geolocation API (like ip-api.com) to fetch country, city, and ISP. Include error handling for invalid IPs, API failures, and JSON parsing issues, and display results clearly.

Code :



```
[6]
0s
def get_geolocation_data(ip_address):
    """
    Fetches geolocation data for a given IP address using ip-api.com.

    Args:
        ip_address (str): The IP address to query.

    Returns:
        dict: A dictionary containing geolocation data if successful, None otherwise.
    """
    # 1. Validate IP address format
    try:
        ipaddress.ip_address(ip_address)
    except ValueError:
        print(f"Error: Invalid IP address format: '{ip_address}'")
        return None

    # 2. Construct API request URL
    # ip-api.com offers a free plan with rate limits for non-commercial use.
    # The 'json' path returns the data in JSON format.
    base_url = f"http://ip-api.com/json/{ip_address}"

    print(f"\nAttempting to fetch geolocation data for IP: {ip_address}...")

    try:
        # 3. Make the API call with a timeout
        response = requests.get(base_url, timeout=5)
        response.raise_for_status()  # Raise an HTTPError for bad responses (4xx or 5xx)
```

Gemini

Generate a Python script that takes an IP address as input, validates its format, and uses a Geolocation API (like ip-api.com) to fetch country, city, and ISP. Include error handling for invalid IPs, API failures, and JSON parsing issues, and display results clearly.

Certainly! Here's the Python script to fetch geolocation data for an IP address, including IP validation, API calls with `ip-api.com`, and robust error handling for various issues. Please make sure you have the `requests` library installed (`pip install requests`) if you encounter any import errors.

The script has successfully imported the required libraries, defined the `get_geolocation_data` function, and executed it with the IP

Test with a private IP address like 192.168.1.1

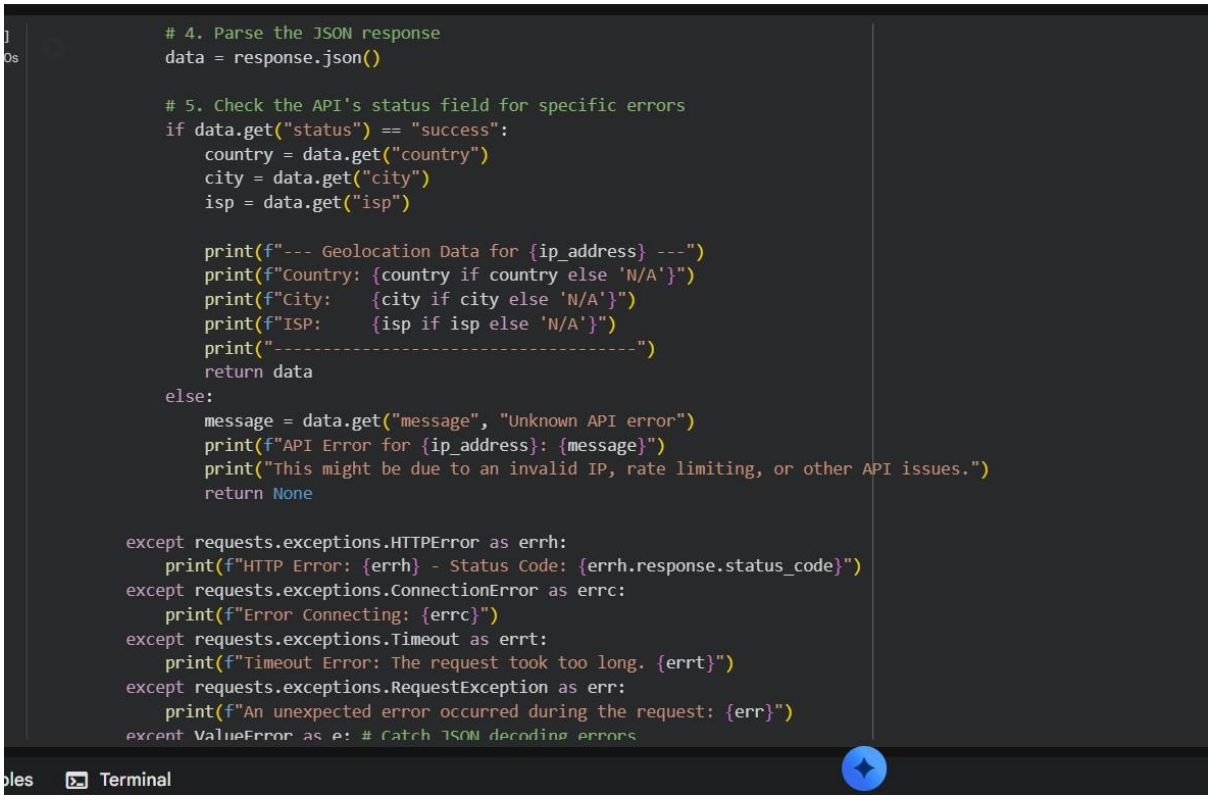
Extract the 'country' and 'isp' from the results

Add documentation to the get_geolocation_data function

What can I help you build?

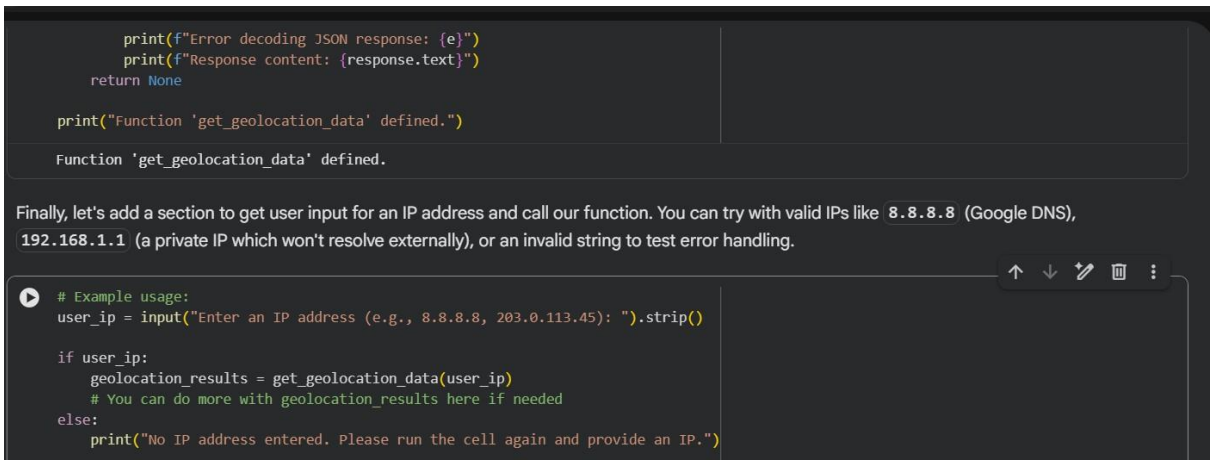
Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. Learn more



```
            # 4. Parse the JSON response
            data = response.json()

            # 5. Check the API's status field for specific errors
            if data.get("status") == "success":
                country = data.get("country")
                city = data.get("city")
                isp = data.get("isp")

                print(f"--- Geolocation Data for {ip_address} ---")
                print(f"Country: {country if country else 'N/A'}")
                print(f"City:    {city if city else 'N/A'}")
                print(f"ISP:     {isp if isp else 'N/A'}")
                print("-------------------------------------")
                return data
            else:
                message = data.get("message", "Unknown API error")
                print(f"API Error for {ip_address}: {message}")
                print("This might be due to an invalid IP, rate limiting, or other API issues.")
                return None

        except requests.exceptions.HTTPError as errh:
            print(f"HTTP Error: {errh} - Status Code: {errh.response.status_code}")
        except requests.exceptions.ConnectionError as errc:
            print(f"Error Connecting: {errc}")
        except requests.exceptions.Timeout as errt:
            print(f"Timeout Error: The request took too long. {errt}")
        except requests.exceptions.RequestException as err:
            print(f"An unexpected error occurred during the request: {err}")
        except ValueError as e: # Catch JSON decoding errors
```

ples Terminal



```
            print(f"Error decoding JSON response: {e}")
            print(f"Response content: {response.text}")
        return None

print("Function 'get_geolocation_data' defined.")
```

```
Function 'get_geolocation_data' defined.
```

Finally, let's add a section to get user input for an IP address and call our function. You can try with valid IPs like `8.8.8.8` (Google DNS), `192.168.1.1` (a private IP which won't resolve externally), or an invalid string to test error handling.

```
# Example usage:
user_ip = input("Enter an IP address (e.g., 8.8.8.8, 203.0.113.45): ").strip()

if user_ip:
    geolocation_results = get_geolocation_data(user_ip)
    # You can do more with geolocation_results here if needed
else:
    print("No IP address entered. Please run the cell again and provide an IP.")
```

Output :

```
Enter an IP address (e.g., 8.8.8.8, 203.0.113.45): 8.8.8.8

Attempting to fetch geolocation data for IP: 8.8.8.8...
--- Geolocation Data for 8.8.8.8 ---
Country: United States
City:    Ashburn
ISP:     Google LLC
------------------------------------
```

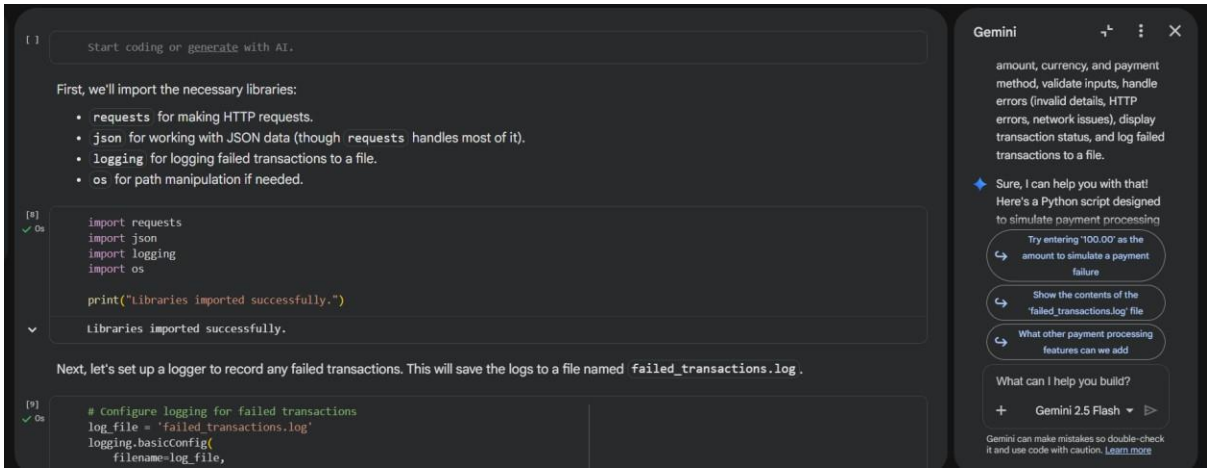
Explanation :

This script takes an IP address, checks if it is valid, and then calls a Geolocation API to get details like country, city, and ISP. It also handles errors such as invalid IP format, API failure, and incorrect JSON response to ensure the program runs safely.

TASK – 05

Prompt : Generate a Python script that simulates payment processing using a sandbox Payment API. It should send a POST request with amount, currency, and payment method, validate inputs, handle errors (invalid details, HTTP errors, network issues), display transaction status, and log failed transactions to a file.

Code :



```
    format='%(asctime)s - %(levelname)s - %(message)s'
)

logger = logging.getLogger(__name__)

print(f"Logging configured. Failed transactions will be saved to '{log_file}'.")
```

```
Logging configured. Failed transactions will be saved to 'failed_transactions.log'.
```

Now, we'll define a function `validate_payment_input` to ensure the amount, currency, and payment method provided by the user are in a valid format before attempting to process the payment.

```
def validate_payment_input(amount, currency, payment_method):
    """
    Validates the payment input details.

    Args:
        amount (str): The payment amount as a string.
        currency (str): The currency code as a string.
        payment_method (str): The payment method as a string.

    Returns:
        bool: True if inputs are valid, False otherwise.
    """
    # Validate amount
    try:
        float_amount = float(amount)
        if float_amount <= 0:
            print("Error: Amount must be a positive number.")
            return False
    except ValueError:
        print("Error: Invalid amount. Please enter a numerical value.")
        return False

    # Validate currency
    if not isinstance(currency, str) or len(currency) != 3 or not currency.isalpha():
        print("Error: Invalid currency. Please use a 3-letter alphabetic code (e.g., 'USD').")
        return False

    # Example: Check against a list of allowed currencies
    allowed_currencies = ['USD', 'EUR', 'GBP', 'JPY']
    if currency.upper() not in allowed_currencies:
        print(f"Error: Unsupported currency '{currency}'. Allowed currencies are: {', '.join(allowed_currencies)}.")
        return False

    # Validate payment method
    allowed_methods = ['credit_card', 'paypal', 'bank_transfer']
    if payment_method.lower() not in allowed_methods:
        print(f"Error: Invalid payment method '{payment_method}'. Allowed methods are: {', '.join(allowed_methods)}.")
        return False

    return True

print("Function 'validate_payment_input' defined.")
```

```
Function 'validate_payment_input' defined.
```

```
1]
 0s
def process_payment(amount, currency, payment_method):
    """
    Simulates processing a payment using a sandbox API.

    Args:
        amount (str): The payment amount.
        currency (str): The currency code.
        payment_method (str): The payment method.

    Returns:
        dict: A dictionary containing the transaction status and details, or None if failed.
    """
    # 1. Input Validation
    if not validate_payment_input(amount, currency, payment_method):
        return None

    # Convert amount to float after validation
    float_amount = float(amount)

    # Mock sandbox API endpoint (httpbin.org/post just reflects the POST data)
    # In a real scenario, this would be a payment gateway's sandbox URL.
    sandbox_api_url = "https://httpbin.org/post"

    # 3. Construct payload for the API request
    payload = {
        'amount': float_amount,
        'currency': currency.upper(),
        'payment_method': payment_method.lower(),
        'description': f"Payment for {float_amount} {currency.upper()}"
```

```
        'payment_method': payment_method.lower(),
        'description': f"Payment for {float_amount} {currency.upper()}"
    }

    headers = {
        'Content-Type': 'application/json'
    }

    print(f"\nAttempting to process payment for {float_amount} {currency.upper()} via {payment_method.lower()}...")

    try:
        # 4. Send POST request
        # Use a timeout to prevent hanging indefinitely
        response = requests.post(sandbox_api_url, headers=headers, data=json.dumps(payload), timeout=10)
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

        # 5. Parse JSON response
        api_response = response.json()

        # --- Simulate payment gateway response ---
        # httpbin.org/post simply returns the request data, so we'll simulate
        # a success/failure based on a simple condition for demonstration.
        # In a real API, 'api_response' would contain the actual transaction status.

        # For demonstration: if amount is 100.0, simulate a failure
        if float_amount == 100.0:
            transaction_status = {
                'status': 'failed',
                'message': 'Simulated: Insufficient funds or card declined.',
                'transaction id': None,
```

```
[11]
✓ 0s
                    'raw_api_response': api_response # include raw response for debugging
                }
                print("Transaction Status: FAILED")
                print(f"Message: {transaction_status['message']}")
                logger.error(f"Failed transaction: Amount={float_amount}, Currency={currency}, Method={payment_method}, Error={transactio
                return transaction_status
            else:
                # Simulate a successful transaction ID
                transaction_id = f"txn_{os.urandom(4).hex()}"
                transaction_status = {
                    'status': 'success',
                    'message': 'Payment processed successfully.',
                    'transaction_id': transaction_id,
                    'raw_api_response': api_response
                }
                print("Transaction Status: SUCCESS")
                print(f"Transaction ID: {transaction_id}")
                print(f"Message: {transaction_status['message']}")
                return transaction_status

        except requests.exceptions.HTTPError as errh:
            error_msg = f"HTTP Error: {errh} - Status Code: {errh.response.status_code}"
            print(error_msg)
            logger.error(f"HTTP Error during payment: Amount={float_amount}, Currency={currency}, Method={payment_method}, Error={error_m
        except requests.exceptions.ConnectionError as errc:
            error_msg = f"Connection Error: {errc} - Could not connect to the API."
            print(error_msg)
            logger.error(f"Connection Error during payment: Amount={float_amount}, Currency={currency}, Method={payment_method}, Error={e
        except requests.exceptions.Timeout as errt:
```

```
            print(error_msg)
            logger.error(f"Timeout Error during payment: Amount={float_amount}, Currency={currency}, Method={payment_method}, Error={erro
        except requests.exceptions.RequestException as err:
            error_msg = f"An unexpected Request error occurred: {err}"
            print(error_msg)
            logger.error(f"Unexpected Request Error during payment: Amount={float_amount}, Currency={currency}, Method={payment_method},
        except json.JSONDecodeError as e:
            error_msg = f"JSON Decoding Error: Could not parse API response. {e}"
            print(error_msg)
            print(f"API raw response: {response.text}")
            logger.error(f"JSON Error during payment: Amount={float_amount}, Currency={currency}, Method={payment_method}, Error={error_m
        except Exception as e:
            error_msg = f"An unexpected error occurred: {e}"
            print(error_msg)
            logger.error(f"General Error during payment: Amount={float_amount}, Currency={currency}, Method={payment_method}, Error={erro

        return None

print("Function 'process_payment' defined.")
```

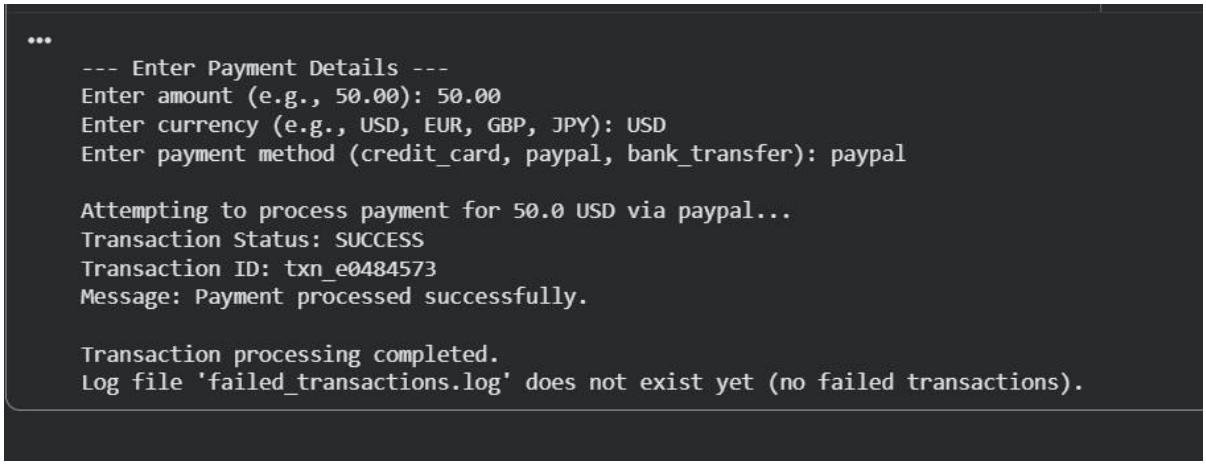
```
Function 'process_payment' defined.
```

Finally, let's add an example usage section where you can input payment details. Try these scenarios:

- **Successful payment:** Amount: `50.00`, Currency: `USD`, Payment Method: `credit_card`
- **Simulated failed payment:** Amount: `100.00`, Currency: `USD`, Payment Method: `credit_card` (This specific amount is hardcoded to trigger a simulated failure for logging demonstration).

```
# Example Usage:
print("\n--- Enter Payment Details ---")
user_amount = input("Enter amount (e.g., 50.00): ").strip()
user_currency = input("Enter currency (e.g., USD, EUR, GBP, JPY): ").strip().upper()
user_payment_method = input("Enter payment method (credit_card, paypal, bank_transfer): ").strip().lower()

# Process the payment
transaction_result = process_payment(user_amount, user_currency, user_payment_method)

if transaction_result:
    print("\nTransaction processing completed.")
    # You can access details like transaction_result['status'], transaction_result['transaction_id']
else:
    print("\nTransaction processing failed due to input validation or an error.")

# Optional: Display the content of the failed_transactions.log file
if os.path.exists(log_file):
    print(f"\n--- Content of '{log_file}' ---")
    with open(log_file, 'r') as f:
        print(f.read())
else:
    print(f"Log file '{log_file}' does not exist yet (no failed transactions).")
```

Output :



Explanation : This script sends payment details (amount, currency, method) to a sandbox API, checks if inputs are valid, and processes the transaction. It handles errors like invalid data, server issues, or network failures, shows the payment status, and logs failed transactions to a file.